Task1

1-1

```
CREATE PROCEDURE CheckAndUpdatePassword(
    IN p_user_name VARCHAR(50),
    IN p_pwd VARCHAR(20),
    IN p_new_pwd VARCHAR(20),
    IN p_action INT
)
BEGIN
    DECLARE pwd_match INT DEFAULT 0;

    SELECT COUNT(*)
    INTO pwd_match
    FROM account
    WHERE user_name = p_user_name AND pwd = p_pwd;

    IF p_action = 1 THEN
        SELECT pwd_match AS 'password_match';
    ELSEIF p_action = 2 THEN
        IF pwd_match = 1 THEN
            IF p_new_pwd REGEXP '^[a-zA-Z0-9]{4,10}$' THEN
                UPDATE account SET pwd = p_new_pwd WHERE user_name = p_user_name;
                SELECT TRUE AS 'password_updated';
            ELSE
                SELECT FALSE AS 'password_updated';
            END IF;
        ELSE
            SELECT FALSE AS 'password_updated';
        END IF;
    END IF;
END$$

DELIMITER ;
```

Statistics 1 | Results 2

SELECT TRUE AS 'pass | Enter a SQL expression to filter results (use Ctrl+Space)

	password_updated
1	1

1-2

```sql
DELIMITER $$

CREATE PROCEDURE BorrowBook(
    IN p_user_name VARCHAR(50),
    IN p_isbn VARCHAR(20)
)
BEGIN
    DECLARE book_count INT DEFAULT 0;
    DECLARE already_borrowed INT DEFAULT 0;

    SELECT book_cnt INTO book_count FROM tusuguan WHERE isbn = p_isbn;
    SELECT COUNT(*) INTO already_borrowed FROM recode
    WHERE user_name = p_user_name AND isbn = p_isbn AND return_time IS NULL;

    IF book_count <= 0 OR already_borrowed > 0 THEN
        SELECT FALSE AS can_borrow;
    ELSE
        BEGIN
            INSERT INTO recode (user_name, isbn, rent_time)
            VALUES (p_user_name, p_isbn, CURDATE());

            UPDATE tusuguan SET book_cnt = book_cnt - 1 WHERE isbn = p_isbn;
        END;

        SELECT TRUE AS can_borrow;
    END IF;
END$$

DELIMITER ;
```

Statistics 1 | Results 2

SELECT TRUE AS can_l | Enter a SQL expression to filter results (use Ctrl+Space)

	can_borrow
1	1

1-3

```
DELIMITER $$

CREATE PROCEDURE ReturnBook(
    IN p_user_name VARCHAR(50),
    IN p_isbn VARCHAR(20)
)
BEGIN
    DECLARE existing_record INT;

    SELECT COUNT(*) INTO existing_record
    FROM recode
    WHERE user_name = p_user_name AND isbn = p_isbn AND return_time IS NULL;

   IF existing_record = 0 THEN
        SELECT FALSE AS success;
    ELSE
        -- Update the return time in the borrowing record
        UPDATE recode
        SET return_time = CURDATE()
        WHERE user_name = p_user_name AND isbn = p_isbn AND return_time IS NULL;

        -- Increase the book count in tusuguan table
        UPDATE tusuguan
        SET book_cnt = book_cnt + 1
        WHERE isbn = p_isbn;

        SELECT TRUE AS success;
    END IF;
END$$

DELIMITER ;
```

Statistics 1

ame	Value
ueries	3
pdated Rows	0
ecute time (ms)	13
tch time (ms)	0
tal time (ms)	13

1-4

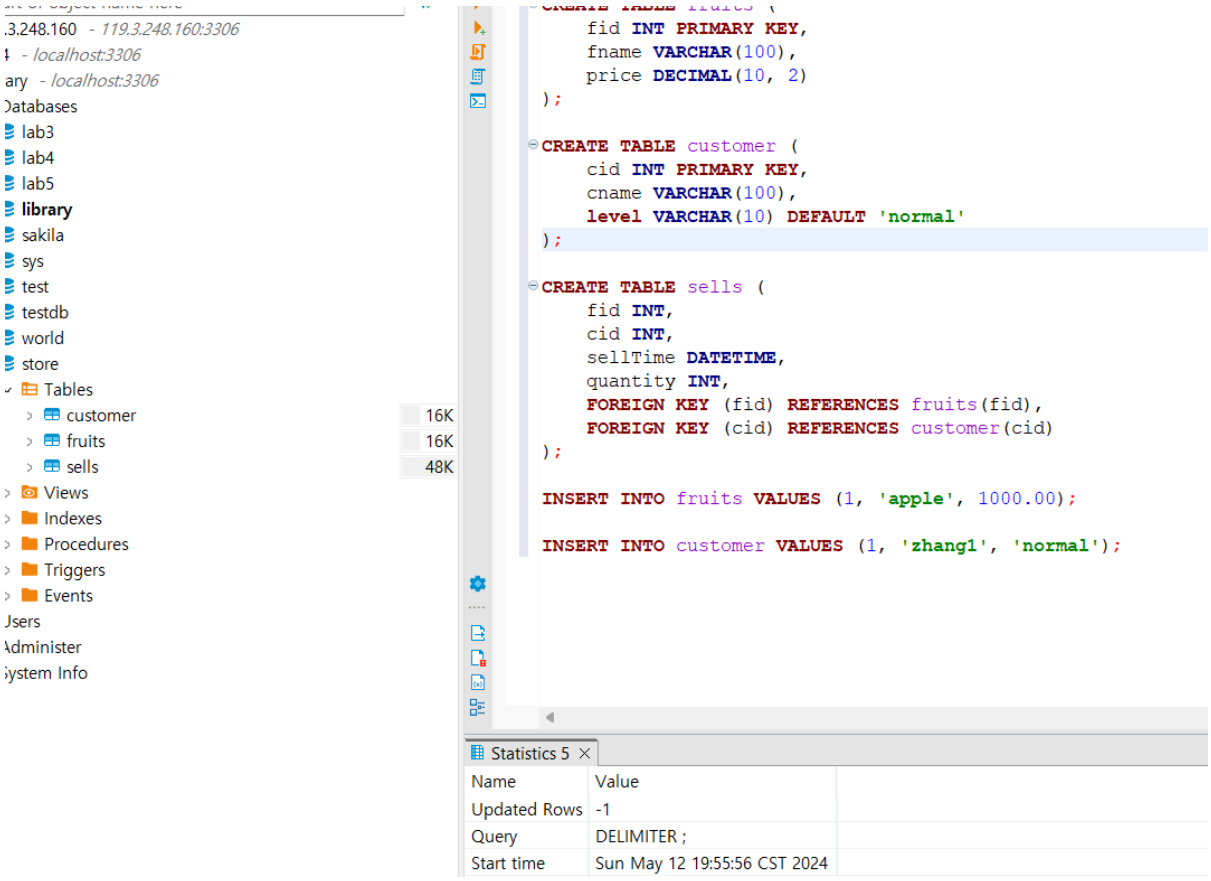
```
DELIMITER $$

CREATE PROCEDURE GetBorrowRecords(
    IN p_user_name VARCHAR(50)
)
BEGIN
    SELECT user_name, isbn, end_time
    FROM recode
    WHERE user_name = p_user_name AND return_time IS NULL;
END$$

DELIMITER ;
```

Statistics 2

Name	Value
Updated Rows	-1
Query	DELIMITER ;
Start time	Sun May 12 17:46:59 CST 2024
Finish time	Sun May 12 17:46:59 CST 2024

Task2

2-1


.3.248.160 - 119.3.248.160:3306
- localhost:3306
ary - localhost:3306
Databases
lab3
lab4
lab5
library
sakila
sys
test
testdb
world
store
Tables
customer 16K
fruits 16K
sells 48K
Views
Indexes
Procedures
Triggers
Events
Users
Administer
System Info
fid INT PRIMARY KEY,
fname VARCHAR(100),
price DECIMAL(10, 2)
);
CREATE TABLE customer (
cid INT PRIMARY KEY,
cname VARCHAR(100),
level VARCHAR(10) DEFAULT 'normal'
);
CREATE TABLE sells (
fid INT,
cid INT,
sellTime DATETIME,
quantity INT,
FOREIGN KEY (fid) REFERENCES fruits(fid),
FOREIGN KEY (cid) REFERENCES customer(cid)
);
INSERT INTO fruits VALUES (1, 'apple', 1000.00);
INSERT INTO customer VALUES (1, 'zhang1', 'normal');
Statistics 5
Name Value
Updated Rows -1
Query DELIMITER ;
Start time Sun May 12 19:55:56 CST 2024

2-2

```mysql
DELIMITER $$

CREATE TRIGGER check_fid_exist
BEFORE INSERT ON sells
FOR EACH ROW
BEGIN
    IF NOT EXISTS (SELECT 1 FROM fruits WHERE fid = NEW.fid) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '该水果数据不存在';
    END IF;
end;$$

DELIMITER ;
```

Statistics 5

Name	Value
Updated Rows	-1
Query	DELIMITER ;
Start time	Sun May 12 19:55:56 CST 2024

2-3

```
DELIMITER $$

CREATE TRIGGER check_cid_exist
BEFORE INSERT ON sells
FOR EACH ROW
BEGIN
    IF NOT EXISTS (SELECT 1 FROM customer WHERE cid = NEW.cid) THEN
        INSERT INTO customer (cid, cname, level) VALUES (NEW.cid, NULL, 'normal');
    END IF;
END$$

DELIMITER ;
```

Statistics 2

Name	Value
Updated Rows	-1
Query	DELIMITER ;
Start time	Sun May 12 20:00:00 CST 2024

2-4

```
DELIMITER $$

CREATE TRIGGER triADD
AFTER INSERT ON sells
FOR EACH ROW
BEGIN
    DECLARE total DECIMAL(10,2);
    SELECT SUM(price * quantity) INTO total
    FROM sells JOIN fruits ON sells.fid = fruits.fid
    WHERE sells.cid = NEW.cid;

    IF total >= 20000 THEN
        UPDATE customer SET level = 'SVIP' WHERE cid = NEW.cid;
    ELSEIF total >= 10000 THEN
        UPDATE customer SET level = 'VIP' WHERE cid = NEW.cid;
    ELSE
        UPDATE customer SET level = 'normal' WHERE cid = NEW.cid;
    END IF;
END$$

DELIMITER ;
```

Statistics 2

Name	Value
Updated Rows	-1
Query	DELIMITER ;
Start time	Sun May 12 20:03:39 CST 2024
Finish time	Sun May 12 20:03:39 CST 2024

2-5

```
DELIMITER $$

CREATE TRIGGER triDEL
AFTER DELETE ON sells
FOR EACH ROW
BEGIN
    CALL recalculate_customer_level(OLD.cid);
END$$

DELIMITER ;

DELIMITER $$

CREATE TRIGGER triUPT
AFTER UPDATE ON sells
FOR EACH ROW
BEGIN
    CALL recalculate_customer_level(NEW.cid);
END$$

DELIMITER ;
```

Statistics 2

Name	Value
Updated Rows	-1
Query	DELIMITER ;
Start time	Sun May 12 22:45:19 CST 2024

2-6

```sql
INSERT INTO sells VALUES (1, 1, NOW(), 20);
UPDATE sells SET quantity = 30 WHERE cid = 1 AND fid = 1;
DELETE FROM sells WHERE cid = 1 AND fid = 1;
```

Statistics 2

Name	Value
Updated Rows	0
Query	DELETE FROM sells WHERE cid = 1 AND fid = 1
Start time	Sun May 12 22:45:51 CST 2024
Finish time	Sun May 12 22:45:51 CST 2024